

QUANTITATIVE TRADING × ADVERSARIAL ML

QuantAdv FAQs

Everything you need to know about the sandbox — how it works, what it simulates, and how to run it yourself.

23

Questions answered

6

Topic sections

quantadv.vercel.app

github.com/John-Varghese-EH/Quantitative-Trading

SECTION 01

General

Q1

What is QuantAdv, in one sentence?

QuantAdv is a risk-free simulation platform where you can build AI trading strategies, attack those AI models with adversarial machine learning techniques, and test defenses to make them more robust — all without touching real money.

Q2

Is this a real trading platform where I can invest money?

No. QuantAdv is a **paper trading / sandbox** environment. All trades are simulated using real or historical market data, but no real capital is ever placed at risk.

Q3

Who is this project for?

Students, ML/AI researchers, cybersecurity learners, FinTech developers, and anyone who wants hands-on experience with both quantitative trading and adversarial ML — without needing a live brokerage account or real financial risk.

Q4

What markets does it cover?

Stocks, cryptocurrency, and forex, using real-time and historical data pulled via APIs like Yahoo Finance, Alpha Vantage, and Binance.

SECTION 02

Core Concepts

Q5

What is "adversarial machine learning" in this context?

It's the study of how AI models can be intentionally fooled by manipulated inputs. In trading, an attacker could subtly alter market data fed into a model to trick it into making a bad Buy/Sell/Hold decision. QuantAdv lets you simulate these attacks safely to understand and defend against them.

Q6

What attack techniques does the sandbox simulate?

Four core techniques: **FGSM** (Fast Gradient Sign Method), **PGD** (Projected Gradient Descent), **data poisoning**, and **label flipping** — covering both inference-time and training-time attacks.

Core Concepts

Q7

What defense techniques are available?

Adversarial training (training on attack examples), **defensive distillation** (smoothing model outputs), **outlier detection** (flagging suspicious inputs), and **Explainable AI via SHAP** (making model decisions inspectable so tampering is easier to catch).

Q8

What ML models are used for price prediction?

Linear Regression, Random Forest, XGBoost, LSTM (recurrent neural networks), and Transformer-based models — giving a spread from simple baselines to state-of-the-art sequence models.

Q9

What does "Explainable AI (XAI)" add to the platform?

It uses SHAP (SHapley Additive exPlanations) values and feature-importance analysis so users can see *why* a model made a given prediction — which is also how you can visually spot when an adversarial attack has skewed a decision.

Features & Usage

Q10

What can I actually do inside the sandbox?

Predict prices, backtest quantitative strategies, run paper trades, launch adversarial attacks on your own models, apply and test defenses, and monitor everything through portfolio, risk, and security dashboards.

Q11

Can I test my own trading strategy?

Yes — the platform supports building and backtesting custom quantitative strategies (e.g., mean reversion, momentum) alongside its AI-driven models.

Features & Usage

Q12

What does the Admin Dashboard do?

It gives platform-level oversight — managing users, monitoring system-wide activity, and overseeing the security / attack-defense testing pipeline.

Q13

Does it show me how well my model is really doing?

Yes — performance reports cover both trading accuracy metrics (returns, risk-adjusted performance) and security metrics (attack success rate, model robustness after defenses).

SECTION 04

Technical

Q14

What's the tech stack?

Frontend: Next.js 15, React 18, TypeScript, Tailwind CSS, Framer Motion, Recharts / TradingView charts

Backend: FastAPI on Python 3.11

ML: Scikit-learn, TensorFlow, PyTorch, XGBoost

Database: PostgreSQL 15

Infrastructure: Docker Compose for containerized deployment

Q15

How is it deployed?

The Next.js frontend is built for Vercel; the FastAPI backend runs via Gunicorn + Uvicorn workers; PostgreSQL runs as a managed instance (e.g., Supabase or AWS RDS). The whole stack can also run locally with a single **docker-compose up --build**.

Q16

Is the project open source?

Yes — it's licensed under **AGPL-3.0**, meaning any modified version offered as a network service must also make its source code openly available.

Technical

Q17

What external APIs / keys does it need to run?

A PostgreSQL connection string, a Financial Modeling Prep (FMP) API key for market data, a Google Gemini API key (used for a "Red Team" threat-intelligence analyst feature), and Firebase credentials for auth / storage.

SECTION 05

Scope & Limitations

Q18

Does QuantAdv guarantee profitable trading strategies?

No. Its purpose is education and research into AI behavior and security — not investment advice or guaranteed returns. Predictions are simulated and should never be used for real financial decisions.

Q19

Is the data real-time or historical?

Both — it supports live market data feeds as well as historical datasets for backtesting and model training.

Q20

What's currently out of scope but planned?

Reinforcement-learning trading agents, live (real-time) paper trading, news / social-media sentiment analysis, blockchain-based trade verification, federated learning, multi-agent simulations, and deep-RL-based portfolio optimization — all listed under future scope.

SECTION 06

Getting Started

Q21

Where can I try it or see the code?

Live demo: quantadv.vercel.app · Source code: github.com/John-Varghese-EH/Quantitative-Trading

Getting Started

Q22

How do I run it locally?

Clone the repo, copy **.env.example** to **.env** and fill in the required API keys, then run **docker-compose up --build**. The frontend runs at **localhost:3000**, the backend API docs at **localhost:8000/docs**, and pgAdmin at **localhost:5050**.

Q23

Is there test coverage?

Yes — backend tests run via **pytest** (with coverage reporting) and frontend tests via **npm run test**.